

Large Language Models

martes, 21 de abril de 2026 10:55 a. m.

Origin of the term “Language Model”

- The term **language model** emerged in **speech recognition**
- Key figure: **Fred Jelinek**
 - Research group: **IBM Research, Yorktown Heights, New York (USA)**
 - Timeframe: late 1970s–1980s

Original problem:

Given ambiguous acoustic input, choose the **most likely sequence of words**

Formalization:

$$\hat{W} = \underset{W}{\operatorname{argmax}} P(A | W) \quad P(W)$$

- $P(A | W)$: acoustic model

Why was probability a good fit for speech recognition instead of rules?
(uncertainty, ambiguity, noisy input)

Modern definition

A **language model** assigns a probability to a sequence of tokens:

$$P(w_1, w_2, \dots, w_n)$$

Using the chain rule:

$$P(w_1^n) = \prod_{t=1}^n P(w_t | w_1^{t-1})$$

Interpretation:

“How surprised should we be by the next token, given the previous ones?”

MLE = Maximum Likelihood Estimation

Core idea:

Choose model parameters that make the observed training data as probable as possible.

Formally, given data $D = w_1^n$ and parameters θ :

$$\hat{\theta} = \underset{\theta}{\operatorname{argmax}} P(D | \theta)$$

For language modeling:

$$\hat{\theta} = \underset{\theta}{\operatorname{argmax}} \prod_{t=1}^n P_{\theta}(w_t | w_1^{t-1})$$

Because products are inconvenient:

$$\hat{\theta} = \underset{\theta}{\operatorname{argmax}} \sum_{t=1}^n \log P_{\theta}(w_t | w_1^{t-1})$$

N-gram Language Models

Markov approximation

Exact conditioning is impossible → approximate:

$$P(w_t | w_1^{t-1}) \approx P(w_t | w_{t-(n-1)}^{t-1})$$

Examples:

- Bigram:** $P(w_t | w_{t-1})$
- Trigram:** $P(w_t | w_{t-2} w_{t-1})$

Estimating n-gram probabilities

Maximum likelihood estimate (MLE):

$$P(w_t | w_{t-1}) = \frac{\text{count}(w_{t-1}, w_t)}{\text{count}(w_{t-1})}$$

Worked Numerical Example

Corpus:

<s> I like cats </s>

<s> I like dogs </s>

Counts:

- count(I like) = 2
- count(like cats) = 1
- count(like dogs) = 1
- count(like) = 2

Compute:

$$P(\text{cats} | \text{like}) = \frac{1}{2}$$

$$P(dogs | like) = \frac{1}{2}$$

Sparsity problem

- Many n-grams **never appear**
- Leads to zero probabilities

Sentence with unseen bigram $\Rightarrow$ total probability = 0

Smoothing (with Example) (7 min)

Why smoothing is needed

Sentence:

"I like parrots"

If "**like parrots**" **never appeared**, then

$$P(parrots | like) = 0$$

Whole sentence probability becomes zero.

Example: Add-one (Laplace) smoothing

Vocabulary size $V = 5$

Counts:

- $\text{count}(\text{like cats}) = 1$
- $\text{count}(\text{like dogs}) = 1$
- $\text{count}(\text{like parrots}) = 0$
- $\text{count}(\text{like}) = 2$

Without smoothing:

$$P(parrots | like) = 0$$

With add-one:

$$P(w | like) = \frac{\text{count}(\text{like } w) + 1}{\text{count}(\text{like}) + V}$$

[Ecuación]

Key idea :

Take a little probability mass from frequent events to allow unseen ones.

Evaluating Language Models: Perplexity

Definition

$$\text{Perplexity} = P(w_1^n)^{-1/n}$$

Equivalent (easier to compute):

[Ecuación]

Example: Perplexity calculation

Suppose a sentence has 4 words and the model assigns:

[Ecuación]

Perplexity:

[Ecuación]

Interpretation:

PP = 10 $\rightarrow$ On average, the model is choosing among **10 equally likely options** at each step.

PP = 100 $\rightarrow$ much worse model

Lower perplexity = less surprised

"Average branching factor"

Is perplexity defined on training data or test data—and why?

If Model A has lower perplexity than Model B, which one generalizes better?

Why N-grams Aren't Enough

Key limitations :

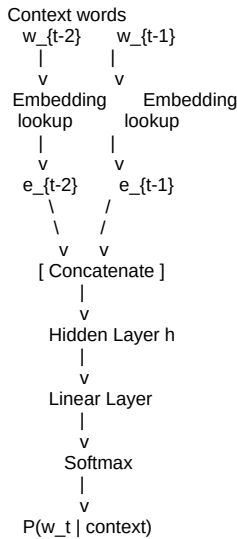
- Fixed context
- Sparsity
- No semantic similarity
- Vocabulary explosion

"Dogs" and "cats" are unrelated unless explicitly seen together.

Neural Language Models

Early neural LM

- Bengio et al. (2003) Université de Montréal, Montreal, Canada



Large Language Models (LLMs)

ELMo (2018)

ELMo = Embeddings from Language Models

- Authors: Peters et al.
- Research group: **Allen Institute for Artificial Intelligence (AI2), Seattle, USA**
 - Collaborators from University of Washington

Key ideas:

- Deep **bidirectional language model**
- Contextual word representations
- Different vectors for same word in different contexts

Example:

- "river bank" vs. "investment bank"

How Do You Train a Large Language Model? Step 1: Data

- Large text corpora:
 - Books, Wikipedia, news, web text
- Scale: billions to trillions of tokens

Step 2: Tokenization

- Subword tokenization (e.g., BPE, WordPiece)
- Handles rare and unseen words efficiently

Step 3: Training Objective (MLE)

Autoregressive objective:

[Ecuación]

Each token is predicted from previous tokens.

Concrete training example

Sentence:

"The cat sat on the mat."

Create training pairs:

input	Target
<s>	The
<s> The	cat
<s> The cat	sat
<s> The cat sat	on
<s> The cat sat on	the
<s> The cat sat on the	mat
<s> ... mat	</s>

Each row contributes one loss term.

Why is next-token prediction considered "self-supervised"?

Step 4: Optimization

- Use **stochastic gradient descent**
- Typically:
 - Mini-batches

- Many epochs (passes over data)
- Compute:
 - Prediction
 - Loss
 - Gradient
 - Parameter update

You do **not** need labeled data beyond raw text.

Step 5: Evaluation

- Held-out test set
- Metric:
 - **Perplexity**
- Lower perplexity = better generalization

Sample uses:

s="The sentiment of the sentence 'I like Pedro Infante' is:" Positive P(Positive|s)>>>
P(Negative|s)

s="Q: Who wrote the book 'Don Quixote'? A:" P(Dickens|s) <<<<< P(Cervantes|s)

"What is the output of the statement 'a' + 'ab' in Python 3? (A) Error (B) aab (C) ab (D) a ab
Answer:"
P(aab|s) P(C | s) P(Error |s) <<<<<<P(B | s)g

